

Sl.no	Topic	Page No.
	Introduction to Computational Complexity	
1.	Introduction	
2.	Time and Space complexity of TMs Time complexity of turing machine Space complexity of turing machine	
3.	Complexity classes	
4.	Introduction to NP-Hardness and NP-Completeness NP-Hardness NP-Completeness NP-Complete problems Formal overview Formal definition of NP-Completeness List of NP-Complete problems Solving NP-Complete problems Boolean satisfiability problem(Sat) The Knapsack problem Unbounded knapsack problem Hamiltonian path problem Randomized algorithm Travelling salesman problem Clique problem	

UNIT-5
INTRODUCTION TO COMPUTATIONAL COMPLEXITY
GROWTH RATE OF FUNCTIONS:

STATEMENT:

When we have two algorithms for the same problem, we may require a comparison between the running times of these two algorithms with this in mind, we study the growth rate of functions defined on the set of natural numbers.

In this section, N describes the set of natural numbers.

Let $f, g : N \rightarrow R^+$ (R^+ being the set of all positive real numbers). We say that $f(n) = O(g(n))$, if there exists positive integers C and N_0 such that,

$$f(n) \leq C g(n) \text{ for all } n \geq N_0.$$

In this case we say “ f ” is the order of g . (Or f is “big oh” of g). $f(n) = O(g(n))$ is not an equation. It expresses a relation between two functions f and g .

For example:

If $f(n) = 4n^3 + 5n^2 + 7n + 3$, we have to prove that $f(n) = O(n^3)$

Solution:

In order to prove that $f(n) = O(n^3)$,

Take $c=5$ and $N_0=10$ then

$$f(n) = 4n^3 + 5n^2 + 7n + 3 \leq 5n^3$$

When $n=10$,

$$f(n) = 5n^2 + 7n + 3 < 5(10^3)$$

$$= 573 < 5(10^3)$$

For $n > 10$,

$$f(n) = 5n^2 + 7n + 3 < 5(n^3)$$

here, $n=11$

$$5(11)^2 + 7(11) + 3 < (11^3)$$

Then,

$$f(n) = O(n^3).$$

Now we can prove that a polynomial algorithm complexity depends on the higher order power of the polynomial.

Statement: If $P(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$, is a polynomial of degree K over Z and a_k an integer and positive as $a_k \geq 1$,

$$P(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$$

$$\div n^k$$

$$\frac{p(n)}{n^k} = \frac{a_k n^k}{n^k} + \frac{a_{k-1} n^{k-1}}{n^k} + \dots + \frac{a_1 n^1}{n^k} + \frac{a_0}{n^k}$$

Since $a_k \geq 1$,

$$a_k + \frac{a_{k-1}}{n} + \dots + \frac{a_1}{n^{k-1}} + \frac{a_0}{n^k} > 0 \quad \text{For all } n \geq N_0.$$

Also

$$\frac{p(n)}{n^k} = a_k + \left(\frac{a_{k-1}}{n} + \dots + \frac{a_1}{n^{k-1}} + \frac{a_0}{n^k} \right)$$

$$\frac{p(n)}{n^k} \leq a_k + 1$$

(Since as $a_{k-1}, a_{k-2}, \dots, a_1, a_0$ are K fixed integers, choose N_0 such that for all $n \geq N_0$ each of the number).

$$\frac{|a_{k-1}|}{n}, \frac{|a_{k-2}|}{n^2}, \dots, \frac{|a_1|}{n^{k-1}}, \frac{|a_0|}{n^k} \quad \text{is less than } 1/k$$

Hence,

Hence,

$$\left| \frac{a_{k-1}}{n} + \frac{a_{k-2}}{n^2} + \frac{a_0}{n^k} \right| < 1$$

$$\frac{p(n)}{n^k} \leq a_k$$

$$P(n) \leq a_{k+1} + n^k \text{ where } c = a_{k+1}.$$

Hence $P(n) = O(n^k)$.

Exponential function:-

An exponential function is a function $q:N \rightarrow N$ defined by $q(n) = a^n$ for some fixed $a > 1$.

When n grows each of n , n^2 , 2^n increases. But the comparison of these functions for specific values of n will indicate the vast difference between the growth rate of the functions.

Growth rate of polynomial and exponential functions:

n	f (n) = n²	g(n)=n²+3n+9	q (n) = 2ⁿ
1	1	13	2
5	25	49	32
10	100	139	1024
50	2500	2659	1.13×10^{15}
100	10000	10309	1.27×10^{30}
1000	1000000	1003009	1.07×10^{301}

From the table, it is easy to say that the function $q(n)$ grows at a very fast rate when compared to $f(n)$ or $g(n)$. In particular the exponential function grows at a very fast rate when compared to $f(n)$ or $g(n)$. In particular the exponential function grows at a very fast rate when compared to any polynomial of large degree.

If f and g are two functions and $f = O(g)$ but $g \neq O(f)$ we say that the growth rate of g is greater rate of f .

The growth rate of any exponential function is greater than that of any polynomials

Let,

$$P(n) = a_k n^k + a_{k-1} n^{k-1} + \dots + a_1 n + a_0$$

And $q(n) = a^n$ for some $a > 1$.

As the growth rate of any polynomial is determined by its term with the highest power, it is enough to prove that:

$$N^k = O(a^n) \text{ and } a^n \neq O(n^k)$$

By L' hospital rule , $\frac{\log n}{n}$ tends to 0 as $N \rightarrow \infty$

If

$$Z(n) = e^{k\left(\frac{\log n}{n}\right)}$$

Then,

$$\begin{aligned} (Z(n))^n &= e^{\left(k\frac{\log n}{n}\right)^n} \\ &= e^{k \log n} \\ &= e^{\log n^k} \\ &= n^k, \end{aligned}$$

As n gets large , $k\left(\frac{\log n}{n}\right)$ tends to 0 and hence $Z(n)$ tends to 0.

So we can choose N_0 such that $Z(n) \leq a$ for all $n \geq N_0$. Hence $n^k = (z(n))^n \leq a^n$

Proving,

$$N^k = O(a^n)$$

To prove that $a^n \neq O(n^k)$, it is enough to show that a^n/n^k is unbounded for large n . but we have proved that

$$N^k \leq a^n \text{ (or) } \frac{a^n}{n^{k+1}} \geq 1.$$

Multiply by n , $n \times \frac{a^n}{n^{k+1}} \geq n$, which means $\frac{a^n}{n^k}$ is unbounded for large values of n .

PRIMITIVE RECURSIVE FUNCTIONS:

We consider automata as the accepting devices initially but automata can act as the computing machines. The problem of finding out whether a given problem is 'solvable' by automata reduces to the evaluation of functions on the set of natural numbers or a given alphabet by mechanical means.

PARTIAL RECURSIVE FUNCTION:

A partial function "f" from X to Y is a rule which assigns to every element of X at most one element of Y.

For e.g. If R denotes the set of all real numbers. The rule "f" from R to itself is given by $f(r) = +\sqrt{r}$ is a partial function since f(r) is not defined as a real number when r is negative.

TOTAL RECURSIVE FUNCTION:

A total function from X and Y is a rule which assigns to every element of X a unique element of Y.

We consider total functions from X^k to X where $X = \{0,1,2,3,\dots\}$ or $X = \{a,b\}^*$. A partial or total function from X^k to X is also called a function of "k" variables and denoted by $f(x_1, x_2, x_3, \dots, x_k)$. for example $f(x_1, x_2) = 2x_1 + x_2$ is a function of two variables.

- a) If $f(1,2)=4$, then 1 and 2 are called arguments and 4 is called the value.
 - b) If $g(w_1, w_2) = w_1 w_2$ is a function of two variables.
 $g(ab, aa) = abaa$, aa are called arguments and abaa is a value of the function.
- We now construct the primitive recursive functions over N and Σ .

Initial Function over N:

The initial functions over N are,

- a) Zero function: Zero function defined by $z(x)=0$.
- b) Successor function "s" defined by $s(x)=x+1$.
- c) Projection function U_i^n defined by $U_i^n(x_1, x_2, \dots, x_n) = x_i$

For eg:

$Z(7)=0$ (zero function)

$S(4)=4+1$ (successor function)
 $=5$

- i. $U_2^3(2,4,7)=4$ (projection function)
- ii. $U_1^3(2,4,7)=2$
- iii. $U_3^3(2,4,7)=7$

Initial Function over Σ :

The initial functions over Σ are given by

$nil(abab) = \epsilon$

$cons a(abab) = aabab$

cons b(abab)=babab
in general form ,the initial functions over Σ are,

nil (x)=&
cons a(x)=ax
cons b(x)=bx

Composition of partial functions:

If $f_1, f_2, f_3, \dots, f_k$ are partial functions of n variables and g is a partial functions of k variables , then the composition of g with $f_1, f_2, \dots, f_k$ is a partial function of n variables defined by

$$G(f_1(x_1, x_2, x_3, \dots, x_n), f_2(x_1, x_2, x_3, \dots, x_n), \dots, f_k(x_1, x_2, x_3, \dots, x_n))$$

If for example f_1, f_2 and f_3 are partial functions of two variables , then the composition of g with f_1, f_2, f_3 is given by $g(f_1(x_1, x_2), f_2(x_1, x_2), f_3(x_1, x_2))$.

For eg:

Let $f_1(x, y)=x+y, f_2(x, y)=2x, f_3(x, y)=xy$ and $g(x, y, z)=x+y+z$ be the functions over

N , Then

$$\begin{aligned} g(f_1(x, y), f_2(x, y), f_3(x, y)) &= g(x+y, 2x, xy) \\ &= x+y+2x+xy \end{aligned}$$

Thus the composition of g with f_1, f_2 and f_3 is given by a function h :

$$h(x, y)=x+y+2x+xy$$

Now we will show that

$f(x, y)=x^y$ is a primitive recursive function.

Soln:

$$f(x, y)=x^y$$

Zero function:

$$\begin{aligned} f(x, y) &= x^y \\ \text{put } y &= 0, \\ f(x, 0) &= x^0 \\ &= 1 \end{aligned}$$

Successor function:

$$\begin{aligned} f(x, y) &= x^y \\ \text{put } y &= y+1 \\ f(x, y+1) &= x^{y+1} \end{aligned}$$

$$=x^y.x$$

$$=f(x,y).x$$

$$=x.f(x,y)$$

This can be represented by projection function as

$$f(x,y+1)=U_1^3(x,y,f(x,y))*U_3^3(x,y,f(x,y))$$

Therefore $f(x,y)$ is primitive recursive.

TIME AND SPACE COMPLEXITY OF COMPLETENESS

Turing machine is defined as the machine which consists of an input tape with the end markers ϕ and $\$$ and k semi infinite tapes where the k is the number of items. The semi infinite storage tape means only one end is closed and the other end is infinite usually at the right side.

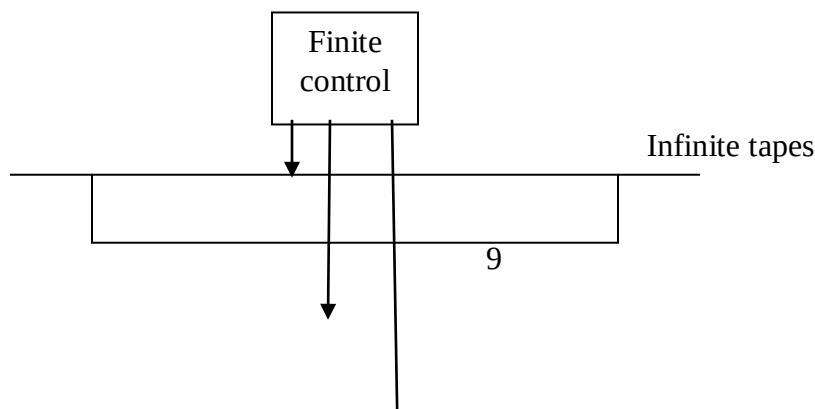
Time Complexity of Turing Machine:

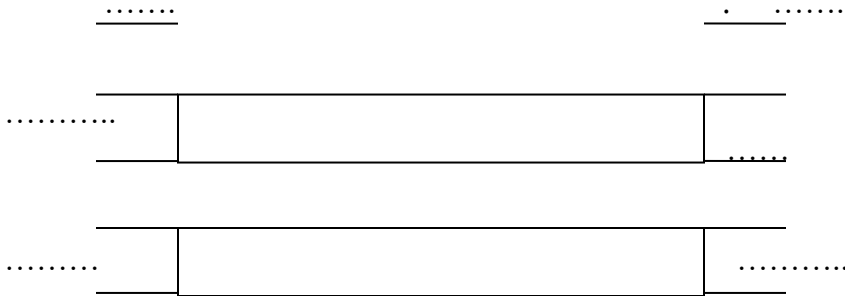
Definition:

For every input word of length n , M (Turing machine) makes at most $T(n)$ moves before halting, then M is said to be a $T(n)$ time-bounded Turing machine, or of time complexity halting, then M is said to be a $T(n)$ time –bounded Turing machine, or of time complexity $T(n)$. The language recognized by M is said to be of time complexity $T(n)$.

Example:

Here the Time Complexity is calculated with respect to the Turing Machine which has k -infinite tapes.
For e.g. $k=3$





No of tapes = No of pointers

Here the first tape is the input tape whereas the other tapes are the storage tape

Consider an language $L=abacaba$ where $w \in a, b$. This language can be written in the form $L=wcw^R$. Here $w=aba$, so n value is 3. Since the length of the string is 3, the Time complexity of the Turing machine is 3 and an extra one is needed for recognizing the intermediate c . So the total time complexity of the Turing machine is given as $T(n)=O(4)$.

Generally the time complexity of the Turing machine is given as

$$T(n)=O(n+1)$$

Here it does not see the alphabets are equal, only the number strings in the right and the left side of the c are to be equal. If the string lengths are equal then n -time needed for recognizing the string and again 1 for the c -intermediate. So it is called as $T(n+1)$ bounded Turing machine.

Growth Function:

Then the growth function is given as

$$f(n) = \frac{n}{n+1}$$

$$f(n) = \frac{n+1}{n+2}$$

Here $n+1$ is the Time Complexity for the string of length n and $n+2$ is the Time Complexity for the string of length $n+1$.

Space Complexity of Turing Machine:

Definition:

The Space Complexity of the Turing machine is indicated as $S(n)$. For every input word of length n , M (Turing Machine) scans at most $S(n)$ cells on any storage tape, then M is said to be an $S(n)$ space-bounded Turing machine or of space complexity $S(n)$. The language recognized by M is said to be of Space complexity $S(n)$.

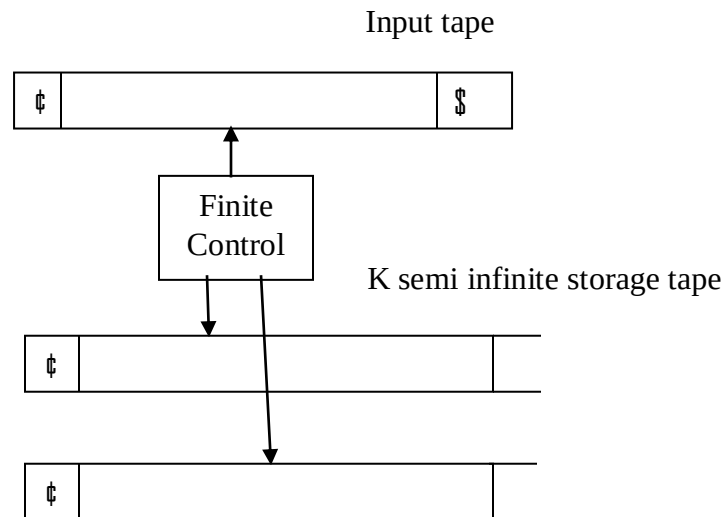
It is also defined as the time required for storing a string of input length n in a k semi infinite tape Turing machine. Here the left side of the Turing machine is bounded and the right side is unbounded.

Example

Here the space complexity of the Turing machine is calculated using the k semi infinite tape as shown above.

Consider an language $L=abcba$ where $w \in a, b$. This language can be written in the form $L=wcwR$. Here $w=ab$, so n value is 2. Since the length of the string is 2, the Space complexity of the Turing machine is 2.

The complexity is given as $\log_2 n$. Here 2 is called the binary counter. The space complexity is given as $\log_2 n$. Here log is used for the comparison.



Space complexity with respect to Least Upper Bound:

Here the LUB is the suprimum.
It can written as

$$\begin{aligned}
 \text{LUB} &= \sup_{n \rightarrow \infty} f(n) \\
 &= \sup_{n \rightarrow \infty} \left[\frac{n}{n+1} \right] \\
 &= \sup_{n \rightarrow \infty} \left[\frac{n}{n(1+1/n)} \right] \\
 &= \sup \left[\frac{1}{1} \right]
 \end{aligned}$$

$$n \rightarrow \infty \quad (1+1/n)$$

Substituting $n=\infty$, we get

$$\text{LUB} = \sup_{n \rightarrow \infty} \left[\frac{1}{(1+1/\infty)} \right]$$

we know that $1/\infty$ value is equal to 0, substituting we get,

$$\text{LUB} = \frac{1}{(1+0)}$$

$$\text{LUB} = 1$$

Space complexity with respect to Greatest Lower Bound:

Here the GLB is the infimum.

It can be written as

$$\text{GLB} = \inf_{n \rightarrow \infty} f(n)$$

$$= \inf_{n \rightarrow \infty} \left[\frac{n}{n+1} \right]$$

$$= \inf_{n \rightarrow \infty} \left[\frac{n}{n(1+1/n)} \right]$$

$$= \inf_{n \rightarrow \infty} \left[\frac{1}{(1+1/n)} \right]$$

Substituting $n=\infty$, we get

$$\text{GLB} = \inf_{n \rightarrow \infty} \left[\frac{1}{(1+1/\infty)} \right]$$

we know that $1/\infty$ value is equal to 0, substituting we get,

$$\text{GLB} = \left[\frac{1}{(1+0)} \right]$$

$$\text{GLB} = 1$$

COMPLEXITY OF CLASSES:

DSPACE(S(n))=deterministic Space complexity(S(n))

NSPACE(S(n))=nondeterministic Space complexity(S(n))

DTIME(T(n))=deterministic Time complexity(T(n))

NTIME(T(n))=nondeterministic Time complexity(T(n))

INTRODUCTION TO NP-HARDNESS AND NP-COMPLETENESS:

NP – complete problems:

In computational complexity theory, the complexity class **NP-complete** (abbreviated **NP-C** or **NPC**, with **NP** standing for nondeterministic polynomial time), is a class of problems having two properties.

- Any given solution to the problem can be verified quickly (in polynomial time); the set of problems with this property is called NP.
- If the problem can be **solved** quickly (in polynomial time), then so can every problem in NP.

Although any given solution to such a problem can be verified quickly, there is no known efficient way to locate a solution in the first place; indeed, the most notable characteristic of NP-complete problems is that no fast solution to them is known. That is, the time required to solve the problem using any currently known algorithm increases very quickly as the size of the problem grows. As a result, the time required to solve even moderately large versions of many of these problems easily reaches into the billions or trillions of years, using any amount of computing power available today. As a consequence, determining whether or not it is possible to solve these problems quickly is one of the principal unsolved problems in computer science today.

While a method for computing the solutions to NP-complete problems using a reasonable amount of time remains undiscovered, computer scientists and programmers still frequently encounter NP-complete problems. An expert programmer should be able to recognize an NP-complete problem so that he or she does not unknowingly waste time trying to solve a problem which so far has eluded generations of computer scientists. Instead, NP-complete problems are often addressed by using approximation algorithms.

Formal overview:

NP-complete is a subset of NP, the set of all decision problems whose solutions can be verified in polynomial time; NP may be equivalently defined as the set of decision problems that can be solved in polynomial time on a nondeterministic Turing machine. A problem p in NP is also in NPC if and only if every other problem in NP can be transformed into p in polynomial time. NP-complete can also be used as an adjective; problems in the class NP complete are known as NP-complete problems.

NP-complete problems are studied because the ability to quickly verify solutions to a problem (NP) seems to correlate with the ability to quickly solve that problem (P). It is not known whether every problem in NP can be quickly solved. This is called the $P=NP$ problem. But if any single problem in NP complete can be solved quickly, then every problem in NP can also be quickly solved, because the definition of an NP-complete problem states that every problem in NP must be quickly reducible to every problem in NP-complete (that is it can be reduced in polynomial time). Because of this, it is often said that the NP-complete problems are harder or more difficult than NP problems in general.

Formal definition of NP-completeness:

A decision problem C is NP-complete if:

1. C is in NP, and
2. Every problem in NP is reducible to C in polynomial time.

C can be shown to be in NP by demonstrating that a candidate solution to C can be verified in polynomial time.

A problem K is reducible to C if there is a polynomial-time many-one reduction, a deterministic algorithm which transforms any instance $k \in K$ into an instance $c \in C$, such that the answer to c is yes if and only if the answer to k is yes. To prove that an NP complete C is in fact NP-complete problem it is sufficient to show that an already known NP-complete problem reduces to C .

Note that a problem satisfying condition 2 is said to be NP-hard, whether or not it satisfies condition 1.

LIST OF NP-COMPLETE PROBLEMS:-

Some NP-complete problems, indicating the reductions typically used to prove their NP-completeness.

An interesting example is the graph isomorphism problem, the graph theory problem of determining whether a graph isomorphism exists between two graphs. Two graphs are isomorphic if one can be transformed into other simply by remaining vertices.

Consider these two problems:

- Graph Isomorphism: Is graph G_1 isomorphic to graph G_2 ?
- Subgraph Isomorphism: Is graph G_1 isomorphic to a subgraph of graph G_2 ?

The subgraph isomorphism problem is NP-complete. The graph isomorphism problem is suspected to be neither in P nor NP-complete, though it is obviously in NP. This is an example of a problem that is thought to be hard, but isn't thought to be NP-complete.

The easiest way to prove that some new problem is NP-complete is first to prove that it is in NP, and then to reduce some known NP-complete problem to it. Therefore, it is useful to know a variety of NP-complete problems. The list below contains some well-known problems that are NP-complete when expressed as decision problems.

- Boolean satisfiability problem (Sat.).
- Knapsack problem.
- Hamiltonian path problem.
- Travelling salesman problem.
- Subgraph isomorphism problem.
- Subset sum problem.

- Clique problem.
- Vertex cover problem.
- Dominating set problem.
- Graph coloring problem.

SOLVING NP-COMPLETE PROBLEMS:-

At present, all known algorithms for NP-complete problems require time that is super polynomial in the input size, and it is unknown whether there are any faster algorithms.

The following techniques can be applied to solve computational problems in general, and they often give rise to substantially faster algorithms:

- Approximation: Instead of searching for an optimal solution search for “almost” optimal one.
- Randomization: Use randomness to get faster average running time, and allow the algorithm to fail with some small probability.
- Restriction: By restricting the structure of the input (e.g., to planar graphs), faster algorithms are usually possible.
- Parameterization: often there are algorithms if certain parameters of the input are fixed.
- Heuristic: an algorithm that works “reasonably well” in many cases, but for which there is no proof that it is both always fast and always produces a good result. Metaheuristic approaches are often used.

One example of a heuristic algorithm is a suboptimal $O(n \log n)$ greedy coloring algorithm used for graph coloring during the register allocation phase of some compilers a technique called graph-coloring global register allocation. Each vertex is a variable, edges are drawn between variables which are being used to indicate the register assigned to each variable.

BOOLEAN SATISFIABILITY PROBLEM (Sat.):-

Satisfiability is the problem determining if the variables of a given Boolean formula can be assigned in such a way as to make the formula evaluate to TRUE. Equally important is to determine whether no such assignments exist, which would imply that the function expressed by the formula is identically False for all possible variable assignments. In this latter case, we would say that the function is unsatisfiable; otherwise it is satisfiable. To emphasize the binary nature of this problem, it is frequently referred to as Boolean or propositional satisfiability. The shorthand “**SAT**” is also commonly used to denote it, with the implicit understanding that the function and its variables are all binary-valued.

In complexity theory, the **Boolean Satisfiability problem (SAT)** is a decision problem, whose instance is a Boolean expression written using only AND, OR, NOT, variables, and parentheses. The question is: given the expression, is there some assignment of TRUE and FALSE values to the variables that will make the entire expression true? A formula of propositional logic is said to be satisfiable if logical values can be assigned to its variables in a way that makes the formula true. The Boolean satisfiable problem is NP-complete.

A **literal** is either a variable or the negation of a variable, (for example, X is a positive literal and negation of X is a negative literal.)

A **clause** is disjunction of literals.

There are several special cases of the Boolean satisfiability problem in which the formulae are required to be conjunction of clauses (i.e. formulae in conjunctive normal form). Determining the satisfiability of a formula in conjunctive normal form where each clause is limited to at most three literals is NP-complete; this problem is called “**3SAT**”, “**3CNFSAT**”, or “**3-satisfiability**”. Determining the satisfiability of a formula in which each clause is limited to at most two literals is NL-complete; this problem is called “**2SAT**”. Determining the satisfiability of a formula in which each clause is a horn clause (i.e. it contains at most one positive literal) is P-complete; this problem is called **Horn-satisfiability**.

NP-COMPLETENESS:-

SAT was the first known NP-complete problem, as proved by Stephen Cook in 1971. Until that time, the concept of an NP-complete problem did not even exist. The problem remains NP-complete even if all expressions are written in conjunctive normal form with 3 variables per clause (3-CNF), yielding the 3SAT problem. This means the expression has the form:

(x11 OR x12 OR x13)AND
(x21 OR x22 OR x23)AND
(x31 OR x32 OR x33)AND.....

Where each x is a variable or a negation of a variable, each variable can appear multiple times in the expression.

3-satisfiability:-

3-satisfiability is a special case of k-satisfiability (k-SAT) or simply satisfiability (SAT), when each clause contains exactly k=3 literals. Here is an example, where indicates NOT:

$E = (x_1 \text{ or } \neg x_2 \text{ or } \neg x_3) \text{ and } (x_1 \text{ or } x_2 \text{ or } x_4)$

E has two clauses (denoted by parentheses), four literals (x_1, x_2, x_3, x_4), and k=3 (three literals per clause).

The knapsack Problem

The knapsack problem or Rucksack problem is a problem in combinatorial optimization: given a set of items, each with a weight and a value, determine the number of each item to include in a collection, so that the total weight is less than a given limit and the total value is as large as possible. It derives its name from the problem faced by someone who is constrained by a fixed-size knapsack and must fill it with the most useful items.

The decision problem from of the knapsack problem is the question “can a value of at least V be achieved without exceeding the weight W ?”

Definition

In the following, we have n kinds of items, 1 through n . Each kind of item j has a value p_j and a weight w_j . We usually assume that all values and weights are nonnegative. The maximum weight that we can carry in the bag is W .

The most common formulation of the problem is the **0-1 Knapsack problem**, which restricts the number x_j of copies of each kind of item to zero or one. Mathematically the 0-1 knapsack problem can be formulated as:

Maximize

Subject to

The **bounded Knapsack problem** restricts the number x_j of copies of each kind of item to a maximum integer value b_j . Mathematically the bounded Knapsack problem can be formulated as:

Maximize

Subject to

The **unbounded Knapsack problem** places no upper bound on the number of copies each kind item.

To solve this instance of the decision problem we must determine whether there is a truth value (TRUE or FALSE) we can assign to each of the literals (x_1 through x_4) such that the entire expression is TRUE. In this instance, there is such an assignment

($x_1=\text{TRUE}, x_2=\text{TRUE}, x_3=\text{TRUE}, x_4=\text{TRUE}$), so the answer to this instance is YES. This is one of many possible assignments, with for instance, any set of assignments including $x_1 = \text{TRUE}$ being sufficient. If there were no such assignment(s), the answer would be NO.

Since k-SAT reduces to 3-SAT, and 3-SAT can be proven to be NP-complete, it can be used to prove that other problems are also NP-complete.

Horn-satisfiability

A clause is Horn if it contains at most one positive literal. Such clauses are of interest because they are able to express implication of one variable from a set of other variables. Indeed, one such clause can be rewritten as, that is, if are all true, then y needs to be true as well.

The problem of deciding whether a set of horn clauses is satisfiable is in P. This Problem can indeed be solved by a single step of the Unit propagation, which produces the single minimal (w.r.t. set of literal assigned to true) model of the set of Horn clauses.

A generalization of the clauses of the class of Horn formulae is that of renamable-Horn formulae, which is the set of formulae that can be placed in Horn form by replacing some variables with their respective negation. Checking the existence of such a replacement can be done in linear time; therefore, the satisfiability of such formulae is in P as it can be solved by first performing this replacement and then checking the satisfiability of the resulting Horn formula.

Of particular interest is the special case, of the problem with these properties:

- It is a decision problem,
- It is a 0-1 problem,
- For each kind of item, the weight equals the value: $w_j = p_j$.

Notice that in this special case, the problem is equivalent to this: Given a set of nonnegative integers, does any subset of it add up to exactly W? Or if negative weights are allowed and W is chosen to zero, the problem is: Given a set of integers, does any subset add up to exactly 0? This special case is called the subset sum problem. In the field of cryptography, the term Knapsack problem is often used to refer specifically to the subset sum problem.

COMPUTATIONAL COMPLEXITY:

The knapsack problem is interesting from the perspective of computer science because

- There is a pseudo-polynomial time algorithm using dynamic programming
- There is a fully polynomial-time approximation scheme, which uses the pseudo-polynomial time algorithm as a subroutine.
- The problem is NP-complete to solve exactly, thus it is expected that no algorithm can be both correct and fast (polynomial-time) on all cases.
- Many cases that arise in practice, and “random instances” from some distributions, can nonetheless be solved exactly.
-

UNBOUNDED KNAPSACK PROBLEM:-

If all weights ($w_1, \dots, w_n$ and W) are nonnegative integers, the knapsack problem can be solved in pseudo-polynomial time using dynamic programming. The following describes a dynamic programming solution for the unbounded knapsack problem.

To simplify thing, assume all weights are strictly positive ($w_j > 0$). We wish to maximize total value subject to the constraint that total weight is less than or equal to W . then for each $Y \leq W$, define $A(Y)$ to be the maximum value that can be attained with total weight less than equal or equal to Y . $A(W)$ then is the solution to the problem.

Observe that $A(Y)$ has the following properties:

- $A(0) = 0$ (the sum of zero items, i.e., the summation of the empty set)
- $A(Y) = \max\{p_j + A(Y - w_j) \mid w_j \leq Y\}$

Where p_j is the value of the j th kind of item.

Here the maximum of the empty set is taken to be zero. Tabulating the results from $A(0)$ up through $A(W)$ gives the solution. Since the calculation of each $A(Y)$ involves examining n items, and there are W values of $A(Y)$ to calculate, the running time of the dynamic programming solution is $O(nW)$. Dividing $w_1, \dots, w_n$, W by their greatest common divisor is an obvious way to improve the running time.

The $O(nW)$ complexity does not contradict the fact that the knapsack problem is NP-complete, since W , unlike N , is not polynomial in the length of the input to the problem.

Hamiltonian path problem

In the mathematical field of graph theory, the Hamiltonian path problem and the Hamiltonian cycle problem are problems of determining whether a Hamiltonian path or a Hamiltonian cycle exists in a given graph (whether directed or undirected). Both problems are NP-complete.

Hamiltonian path

A Hamiltonian path (or unceable path), is a path in an undirected graph which visits each vertex exactly once. A Hamiltonian cycle (or Hamiltonian circuit) is a cycle in an undirected graph, which visits each vertex exactly once and also returns to the starting vertex. Determining whether such paths and cycles exist in graphs is the Hamiltonian path problem which is NP-complete.

Examples

- A complete graph with more than two vertices is Hamiltonian
- Every cycle graph is Hamiltonian

There is a simple relation between the two problems. The Hamiltonian path problem for graph G is equivalent to the Hamiltonian cycle problem in a graph H obtained from G by adding a new vertex and connecting it to all vertices of G.

The Hamiltonian cycle problem is a special case of the traveling salesman problem, Obtained by setting the distance between two cities to a finite constant if they are adjacent and infinity otherwise.

The directed and undirected Hamiltonian cycle problems were two of karp's 21 NP-complete problems. Garey and Johnson showed shortly afterwards in 1974 that the directed Hamiltonian cycle problem remains NP- complete for planar graphs and the Undirected Hamiltonian cycle problem remains NP-complete for cubic planar graphs.

Randomized Algorithm

A randomized algorithm for Hamiltonian path that is fast on most graphs is following:

Start from a random vertex and continue if there is a neighbor not visited. If there are no more unvisited neighbors, and the path formed isn't Hamiltonian, pick a neighbor uniformly at random, and rotate using that neighbor as a pivot. (That is, add an edge to that neighbor, and remove one of the existing edges from that neighbor so as not to form a loop.) Then, continue the algorithm at the new end of the path.

TRAVELLING SALESMAN PROBLEM

The Travelling Salesman Problem (TSP) is a problem in combinatorial optimization studied in operations research and theoretical computer science. Given a list of cities and their pair wise distances, the task is to find a shortest possible tour that visits each city exactly once.

As a graph problem.

The TSP can be modeled as a graph, such that cities are the graph's vertices, paths are the graph's edges, and a path's distance is the edge's length. A TSP tour becomes a Hamiltonian cycle, and the optimal TSP tour is the shortest Hamiltonian cycle.

Often, the model is a complete graph (i.e. an edge connects each pair of vertices). If no path exists between two cities, adding an arbitrarily long edge will complete the graph without affecting the optimal tour.

Asymmetric and symmetric.

In the symmetric TSP, the distance between two cities is the same in each direction, forming an undirected graph. This symmetry halves the number of possible solutions. In the asymmetric TSP, paths may not exist in both directions or the distances might be different, forming a directed graph.

With metric distances.

In the metric TSP the intercity distances satisfy the triangle inequality. This can be understood as “no shortcuts”, in the sense that the direct connection from A to B is never longer than the detour via C.

These edge lengths define a metric on the set of vertices. When the cities are viewed as points in the plane, many natural distance functions are metrics.

- In the Euclidian TSP the distance between two cities is the Euclidean distance between the corresponding points.
- In the Rectilinear TSP the distance between two cities is the sum of the differences of their x- and y-coordinates.

Non-metric distances.

Distance measures that do not satisfy the triangle inequality arise in many routing problems. For example, one mode of transportation, such as travel by airplane, may be faster, even though it covers a longer distance.

In its definition, the TSP does not allow cities to be visited twice, but many applications do not need this constraint. In such cases, a symmetric, non-metric instance can be reduced to a metric one. This replaces the original graph with a complete graph in which the inter-city distance C_{ij} is replaced by the shortest path between i and j in the original graph.

CLIQUE PROBLEM

In computational complexity theory, the clique problem is a graph-theoretic NP-complete problem. The problem was one of the Richard Karp’s original 21 problems shown NP-complete in his 1972 paper “Reducibility among Combinatorial Problems”.

A clique in a graph is a set of pair wise adjacent vertices, or in other words, an induced sub graph which is a complete graph. In the graph at the right, vertices 1, 2 and 5 form a clique, because each has an edge to all the others.

Then, the clique problem is the problem of determining whether a graph contains a clique of at least a given size k . Once we have located k or more vertices which form a clique, it’s trivial to verify that they do, which is why the clique problem is

in NP. The corresponding optimization problem, the maximum clique problem, is to find the largest clique in a graph.

The NP-completeness of the clique problem follows trivially from the NP-completeness of the independent set problem, because there is a clique of size at least k if and only if there is an independent set of size at least k in the complement graph. This is easy to see, since if a sub graph is complete; its complement sub graph has no edges at all.

Algorithm

A brute force algorithm to find a clique in a graph is to examine each sub graph with at least k vertices and check to see if it forms a clique. This algorithm is polynomial if k is the number of vertices, or a constant less than this, but not if k is, say, half the number of vertices; the number of possible cliques of size k on a graph of size V is equal to combination of vertices (V) and size k .

A heuristic is to start by considering each node to be a clique of size one, and to merge cliques into larger cliques until there are no more possible merges. Two cliques A and B may be merged if each node in clique A is adjacent to each node in clique B . This requires only linear time (linear in the number of edges), but may fail to find a large clique because two or more parts of the large clique have already been merged with nodes that are not in the clique. The algorithm can be implemented most efficiently using the disjoint-set data structure.

Vertex Cover Problem:

A vertex cover of a graph is a set of vertices such that each edge of graph is incident to at least one vertex of the set.

THE COOK-LEVIN THEOREM:

Once we have one NP – complete problem, we may obtain others by polynomial time reduction from it. However, establishing the first NP-complete problem is more difficult. Now we do so by proving that SAT is NP-complete.

STATEMENT:

SAT is NP-complete.

TO PROVE:

Showing that SAT is in NP is easy, and we do so shortly. The hard part of the proof is showing that any language in NP is polynomial time reducible to SAT.

To do so we construct a polynomial time reduction for each language A in NP to SAT. The reduction for A takes a string w and produces a Boolean formula ϕ that simulates the NP machine for A on input w . If the machine accepts, ϕ has a satisfying assignment that corresponds to the accepting computation. If the machine doesn't accept, no assignment satisfies ϕ . Therefore w is in A if and only if ϕ is satisfiable.

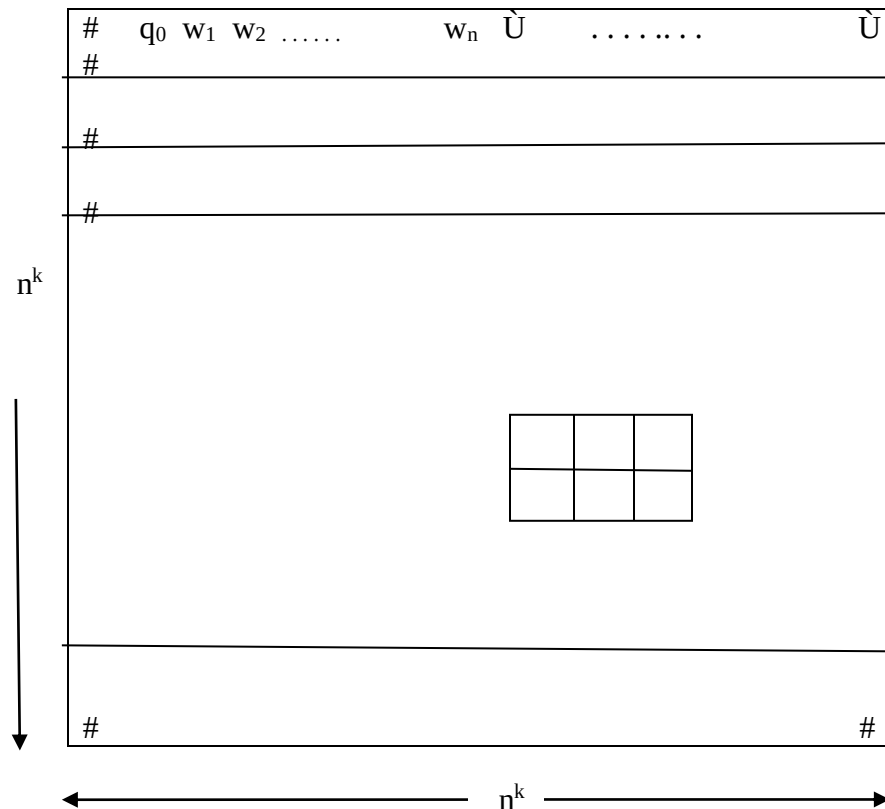
Actually constructing the reduction to work in this way is a conceptually simple task , though we must cope with many details. A Boolean formula may contain the Boolean operations AND, OR and NOT, these operations form the basis for the circuitry used in electronic computers. Hence the fact that we can design a Boolean formula to simulate a Turing machine.

PROOF:

First we show that SAT is in NP. A nondeterministic polynomial time machine can guess an assignment to a given formula ϕ and accept if the assignment satisfies ϕ .

Next, we take any language A in NP and show that A is polynomial time reducible to SAT. Let N be a nondeterministic Turing machine that decides A in n^k time for some constant k. (we assume that N runs in time n^{k-3}).The following notion helps to describe the reduction.

A tableau for N on w is an $n^k \times n^k$ table whose rows are the configurations of a branch of the computation of N on input w ,as shown in the following figure.



A Tableau is an $n^k \times n^k$ table of configurations.

For convenience later we assume that each configuration starts and ends with a # symbol, so the first and last columns of a tableau are all #s. The first row of the tableau is the starting configuration of N on w, and each row follows the previous one according to N's transition function. A tableau is accepting if any row of the tableau is an accepting configuration.

Every accepting tableau for N on w correspond to an accepting computation branch of N on w. Thus the problem of determining whether N accepts w is equivalent to the problem of determining whether an accepting tabuleau for N on w exists.

Now we get to the description of the polynomial time reduction f from A to SAT. On input w, the reduction produces a formula $\emptyset$. We begin by describing the variables of $\emptyset$. Say that Q and Γ are the state set and tape alphabet of N. Let $C = Q \cup \Gamma \cup \{\#\}$. For each I and j between 1 and n^k and for each s in C we have a variable, $x_{i,j,s}$.

Each of the $(n^k)^2$ entries of a tableau is called a cell. The cell is row I and column j is called $\text{cell}[i, j]$ and contains a symbol from C. We represent the contents of the contents of the cells with the variables of $\emptyset$. If $x_{i,j,s}$ takes on the value 1, it means that $\text{cell}[i, j]$ contains an s.

Now we design $\emptyset$ so that a satisfying assignment to the variables does correspond to an accepting tableau for N on q. The formula $\emptyset$ is the AND of four parts $\emptyset_{\text{cell}} \wedge \emptyset_{\text{start}} \wedge \emptyset_{\text{move}} \wedge \emptyset_{\text{accept}}$. We describe each part in turn.

As we mentioned previously, Turing variable $x_{i,j,s}$ on corresponds to placing symbol s in cell $[i, j]$. The first thing we must guarantee in order to obtain a correspondence between an assignment and a tableau is that the assignment turns on exactly one variable for each cell. Formula $\emptyset_{\text{cell}}$ ensures this requirement by expressing it in terms of Boolean operations:

$$\emptyset_{\text{cell}} = \bigwedge_{1 \leq i, j \leq n^k} \left[\left(\bigvee_{s \in C} x_{i,j,s} \right) \wedge \left(\bigwedge_{\substack{s, t \in C \\ s \neq t}} (\overline{x_{i,j,s}} \vee \overline{x_{i,j,t}}) \right) \right]$$

The symbols $\wedge$ and $\vee$ stand for iterated AND and OR. For example, the expression in the preceding formula

$$\bigvee_{s \in C} x_{i,j,s}$$

is shorthand for

$$x_{i,j,s1} \vee x_{i,j,s2} \vee \dots \vee x_{i,j,s1}$$

where $C = \{s1, s2, \dots, s1\}$. Hence $\emptyset_{\text{cell}}$ is actually a large expression that contains a fragment for each cell in the tableau because i and j range from 1 to n^k . The first part of each fragment says that at least one variable is turned on in the corresponding cell. The second part of each fragment says that no more than one variable is turned on (literally, it says

that in each pair of variables , at least one is turned off) in the corresponding cell. These fragments are connected by $\wedge$ operations .

The first part of $\emptyset_{\text{cell}}$ inside the brackets stipulates that at least one variable that is associated to each cell is on , whereas the second part stipulates that no more than one variables is on for each cell. Any assignment to the variables that satisfies $\emptyset$ (and therefore $\emptyset_{\text{cell}}$) must have exactly one variable on for every cell. Thus any satisfying assignment specifies one symbol in each cell of the table. Parts $\emptyset_{\text{start}}$, $\emptyset_{\text{move}}$ and $\emptyset_{\text{accept}}$ ensure that these symbols actually correspond to an accepting tableau as follows.

Formula $\emptyset_{\text{start}}$ ensures that the first row of the table is the starting configuration of N on w by explicitly stipulating that the corresponding variables are on:

$$\emptyset_{\text{start}} = x_{1,1,\#} \wedge x_{1,2,q_0} \wedge x_{1,3,w_1} \wedge x_{1,4,w_2} \wedge \dots \wedge x_{1,n+2,w_n} \wedge x_{1,n+3,\hat{U}} \wedge x_{1,nk,\#}.$$

$\emptyset_{\text{accept}}$ guarantees that an accepting configuration occurs in the tableau. It ensures that q_{accept} , the symbol fir the accept state, appears in one of the cells of the tableau, by stipulating that one of the corresponding variables is on :

$$\emptyset_{\text{accept}} = \bigvee_{1 \leq i, j \leq n^k} x_{i,j,q_{\text{accept}}}$$

Finally, formula $\emptyset_{\text{move}}$ guarantees that each row of the table corresponds to a configuration that legally follows the preceding row's configuration according to N's rules. It does so by ensuring that each 2×3 window of cells is legal. We say that a 2×3 window is legal if that window does not violate the actions specified by N's transition function. In other words, a window is legal if it might appear when one configuration correctly follows another.

For example, say that a, b and c are members of the tape alphabet and q_1 and q_2 are states of N. Assume that when in state q_1 with the heading reading an a , N writes a, b stays in state q_1 and moves right , and that when in state q_1 with the heading a b , N nondeterministic ally either

1. Writes a c , enters q_2 and moves to the left , or
2. Writes an a , enters q_2 and moves to the right.

Expressed formally , $\delta(q_1,a)=\{(q_1,b,R)\}$ and $\delta(q_1,b)=\{(q_2,c,L),(q_2,a,R)\}$. Examples of legal windows for this machine are shown in the following figure.

(a)	(b)	(c)																		
<table border="1"><tr><td>a</td><td>q_1</td><td>b</td></tr><tr><td>q_2</td><td>a</td><td>c</td></tr></table>	a	q_1	b	q_2	a	c	<table border="1"><tr><td>a</td><td>q_1</td><td>b</td></tr><tr><td>a</td><td>a</td><td>q_2</td></tr></table>	a	q_1	b	a	a	q_2	<table border="1"><tr><td>a</td><td>a</td><td>q_1</td></tr><tr><td>a</td><td>a</td><td>b</td></tr></table>	a	a	q_1	a	a	b
a	q_1	b																		
q_2	a	c																		
a	q_1	b																		
a	a	q_2																		
a	a	q_1																		
a	a	b																		
(d)	(e)	(f)																		
<table border="1"><tr><td>#</td><td>b</td><td>a</td></tr><tr><td>#</td><td>b</td><td>a</td></tr></table>	#	b	a	#	b	a	<table border="1"><tr><td>a</td><td>b</td><td>a</td></tr><tr><td>a</td><td>b</td><td>q_2</td></tr></table>	a	b	a	a	b	q_2	<table border="1"><tr><td>b</td><td>b</td><td>b</td></tr><tr><td>c</td><td>b</td><td>b</td></tr></table>	b	b	b	c	b	b
#	b	a																		
#	b	a																		
a	b	a																		
a	b	q_2																		
b	b	b																		
c	b	b																		

Examples of legal windows

In the above figure , windows (a) and (b) are legal because the transition function allows N to move in the indicated way. Window (c) is legal because, with q_1 appearing on the right side of the top row, we don't know what symbol the head is over. That symbol could be an a, and q_1 might change it to a b and move to the right. That possibility would give rise to this window , so it doesn't violate N's rules. Window (d) is obviously legal because the top and bottom are identical, which would occur if the head weren't adjacent to the location of the window. Note that # may appear on the left or right of both the top and bottom rows in a legal window. Window (e) is legal because state q_1 reading a b might have been immediately to the right of the top row, and if would then have moved to the left in state q_2 to appear on the right – hand end of the bottom row. Finally , window (f) is legal because state q_1 might have been immediately to the left of the top row and it might have changed the b to a c and moved to the left.

The window shown in the following figure aren't legal for machine N.

(a)	(b)	(c)																		
<table border="1"> <tr> <td>a</td><td>b</td><td>a</td></tr> <tr> <td>a</td><td>a</td><td>a</td></tr> </table>	a	b	a	a	a	a	<table border="1"> <tr> <td>a</td><td>q_1</td><td>b</td></tr> <tr> <td>q_1</td><td>a</td><td>a</td></tr> </table>	a	q_1	b	q_1	a	a	<table border="1"> <tr> <td>b</td><td>q_1</td><td>b</td></tr> <tr> <td>q_2</td><td>b</td><td>q_2</td></tr> </table>	b	q_1	b	q_2	b	q_2
a	b	a																		
a	a	a																		
a	q_1	b																		
q_1	a	a																		
b	q_1	b																		
q_2	b	q_2																		

Examples of illegal windows

In window (a) the central symbol in the top row can't change because a state wasn't adjacent to it. Window (b) isn't legal because the transition function that the b gets changed to a c but to an a. Window (c) isn't legal; because two states appear in the bottom row.

Part 2:

If the top row of the table is the start configuration and every window in the table is legal, each row of the table is a configuration that legally follows the preceding one.

We prove this claim by considering any two adjacent configurations in the table, called the upper configuration and the lower configuration. In the upper configuration, every cell that isn't adjacent to a state symbol and that doesn't contain the boundary symbol #, is the center top cell in a window whose top row contain no states. Therefore that symbol must appear unchanged in the center bottom of the window. Hence it appears in the same position in the bottom configuration

The window containing the state symbol in the center top cell guarantees that the corresponding three positions are updated consistently with the transition function. Therefore, if the upper configuration is a legal configuration, so is the lower configuration , and the lower configuration follows the upper one according to N's rules.

Now we return to the construction of $\mathcal{O}_{\text{move}}$. It stipulates that all the windows in the tableau are legal. Each window contains six cells, which may be set in a fixed number of ways to yield a legal window. Formula $\mathcal{O}_{\text{move}}$ says that the settings of those six cells must be one of these ways , or

$$\emptyset_{move} = \bigwedge_{1 \leq i \leq n^k, 1 \leq j \leq n^k} (the(i,j) window is legal)$$

we replace the text “the (I,j) window is legal “ in this formula with the following formula. We write the contents of six cells of a window as $a_1, \dots, a_6$.

$$\bigvee_{a_1 \dots a_6 \text{ is a legal window}} (x_{i,j-1,a_1} \wedge x_{i,j+1,a_2} \wedge x_{i,j+1,a_3} \wedge x_{i+1,j,a_4} \wedge x_{i+1,j+1,a_5} \wedge x_{i+1,j+1,a_6})$$

Next we analyze the complexity of the reduction to show that it operates in polynomial time. To do so we examine the size of $\emptyset$. First, we estimate the number of variables it has. Recall that the tableau is an $n^k \times n^k$ table, so it contains n^{2k} cells. Each cell has l variables with it, where l is the number of variables in C . Because l depends only on the TM N and not on the length of the input n , the total number of variables is $O(n^{2k})$.

We estimate the size of each of the parts of $\emptyset$. Formula $\emptyset_{cell}$ contains a fixed-size fragment of the formula for each of the tableau, so its size is $O(n^{2k})$. Formula $\emptyset_{start}$ has a fragment for each cell in the top row, so its size is $O(n^{2k})$. Formula $\emptyset_{start}$ has a fragment for each cell in the top row, so its size is $O(n^k)$. Formulas $\emptyset_{move}$ and $\emptyset_{accept}$ each contain a fixed-size fragment of the formula for each cell of the tableau, so size is $O(n^{2k})$. Thus $\emptyset$'s total size is $O(n^{2k})$. That bound is sufficient for our purposes because it shows that the size of $\emptyset$ is polynomial in n . If it were more than polynomial, the reduction wouldn't have any change of generating it in polynomial time. (Actually our estimates are low by a factor of $O(\log n)$ because each variable has indices that can range up to n^k and so may require $O(\log n)$ symbols to write into the formula, but this additional factor doesn't change the polynomial of the result.)

To see that we can generate the formula in polynomial time, observe its highly repetitive nature. Each component of the formula is composed of many nearly identical fragments, which differ only at the indices in a simple way. Therefore we may easily construct a reduction that produces $\emptyset$ in polynomial time from the input w .

Thus we have concluded the proof of the Cook-Levin theorem, showing that SAT is NP-complete.